



Smart Contract Code Review And Security Analysis Report

CUSTOMER Coinversa
SCOPE Canton DAML Smart Contracts
DATE 17.03.2026

Made in Germany by softstack GmbH

Table of Contents

1. Disclaimer	4
2. About the Project and Company	5
3. Vulnerability & Risk Level	6
4. Auditing Strategy and Techniques Applied	7
5. Metrics	8
5.1 Tested Contract Files	8
5.2 Source Lines & Risk	8
5.3 Capabilities	9
5.4 Dependencies / External Imports	9
5.5 Source Units in Scope	9
6. Scope of Work	10
6.1 Findings Overview	10
6.2 Manual and Automated Vulnerability Test	11
6.2.1 Operator Controls UserIdentity Mutations That Should Belong to the User	11
6.2.2 Core Records Lack Enforced Uniqueness in the Current Canton 3 Design	12
6.2.3 User Cannot Revoke Their Own Terms Acceptance	13
6.2.4 Terms Acceptance Is Not Enforced by the Transaction Workflow	14
6.2.5 Operator-Supplied acceptedAt Timestamp Is Unconstrained	14
6.2.6 Wallet Address Validation Is Mostly Length-Based	15
6.2.7 Transaction Hash Validation Accepts Invalid Pending and EVM Forms	16
6.2.8 LinkWallet Does Not Verify Wallet Ownership	16
6.2.9 sourceWallet Is Not Validated Against walletType in TransactionCommitment	17
6.2.10 termsHash Checks Length but Not Hex Content	17
6.2.11 LinkAuthKey Allows Duplicate Authentication Keys	18
6.2.12 Redeem Does Not Enforce a Transaction-State Transition	19
6.2.13 ArchiveTransaction Allows Unilateral Record Deletion	19
6.2.14 Same-State UpdateStatus Calls Can Still Change blockNumber	20
6.2.15 Statistics Counters Are Not Consistently Monotonic	20
6.2.16 Referrer Can Be Rewritten or Removed Without Constraint	21
6.2.17 UserStatistics Is Not Derived From Transaction Records	22
6.2.18 signedMessage Is Not Bound to Wallet, Terms Version, or Terms Hash	22
6.2.19 ipAddress Is Stored Directly On-Ledger	23
6.2.20 Terms Versions Have No On-Ledger Lifecycle Management	23

7. Executive Summary

25

8. About the Auditor

26

9. Glossary

27

1. Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the client. If you are not the intended receptor of this document, remember that any disclosure, copying or dissemination of it is forbidden.

Version / Date	Description
0.1 (10.03.2026)	Layout
0.5 (14.03.2026)	Manual & Automated Security Testing
0.9 (17.03.2026)	Summary and Recommendation
1.0 (17.03.2026)	Final document

2. About the Project and Company

Company: Coinversa Inc

Address: 1007 North Orange St., 4th Floor, Wilmington, Delaware 19801, USA

Website: <https://www.coinversaa.ai>

GitHub: <https://github.com/Coinversaa>

Coinversa provides infrastructure for on-chain trading, enabling users to analyze markets, coordinate multiple wallets, and execute trades directly from their own custody. The platform indexes clearinghouse state and trader activity in real time to power portfolio visibility, market intelligence, and coordinated execution across wallets.

Trading activity is reconciled through a private identity layer anchored on the Canton Network, ensuring non-custodial execution while maintaining a unified coordination layer across connected wallets. The platform is designed for discretionary traders and systematic strategies, starting with Hyperliquid as the first supported venue.

The audited scope covers the Canton DAML smart contracts that implement user identity management, wallet linking, transaction commitment tracking, terms acceptance, and user statistics — the core on-ledger primitives that underpin Coinversa's identity and reconciliation layer.

3. Vulnerability & Risk Level

Risk represents the probability that a certain source-threat will exploit vulnerability, and the impact of that event on the organization or system. Risk Level is computed based on CVSS version 3.1.

Level	CVSS Score	Vulnerability	Required Action
CRITICAL	9.0 – 10.0	A vulnerability that can disrupt the contract functioning in a number of scenarios, or creates a risk that the contract may be broken.	Immediate action to reduce risk level.
HIGH	7.0 – 8.9	A vulnerability that affects the desired outcome when using a contract, or provides the opportunity to use a contract in an unintended way.	Implementation of corrective actions as soon as possible.
MEDIUM	4.0 – 6.9	A vulnerability that could affect the desired outcome of executing the contract in a specific scenario.	Implementation of corrective actions in a certain period.
LOW	0.1 – 3.9	A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective.	Implementation of certain corrective actions or accepting the risk.
INFORMATIONAL	0.0	A vulnerability that has informational character but is not affecting any of the code.	An observation that does not determine a level of risk.



4. Auditing Strategy and Techniques Applied

Throughout the review process, care was taken to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

The auditing process follows a routine series of steps:

Code review that includes the following:

- Review of the specifications, sources, and instructions provided to softstack to make sure we understand the size, scope, and functionality of the smart contract.
- Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
- Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to softstack describe.

Testing and automated analysis that includes the following:

- Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
- Symbolic execution, which is analysing a program to determine what inputs causes each part of a program to execute.

Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarify, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.

Specific, itemized, actionable recommendations to help you take steps to secure your smart contracts.

5. Metrics

The metrics section gives the reader an overview of the size, quality, flows and capabilities of the codebase.

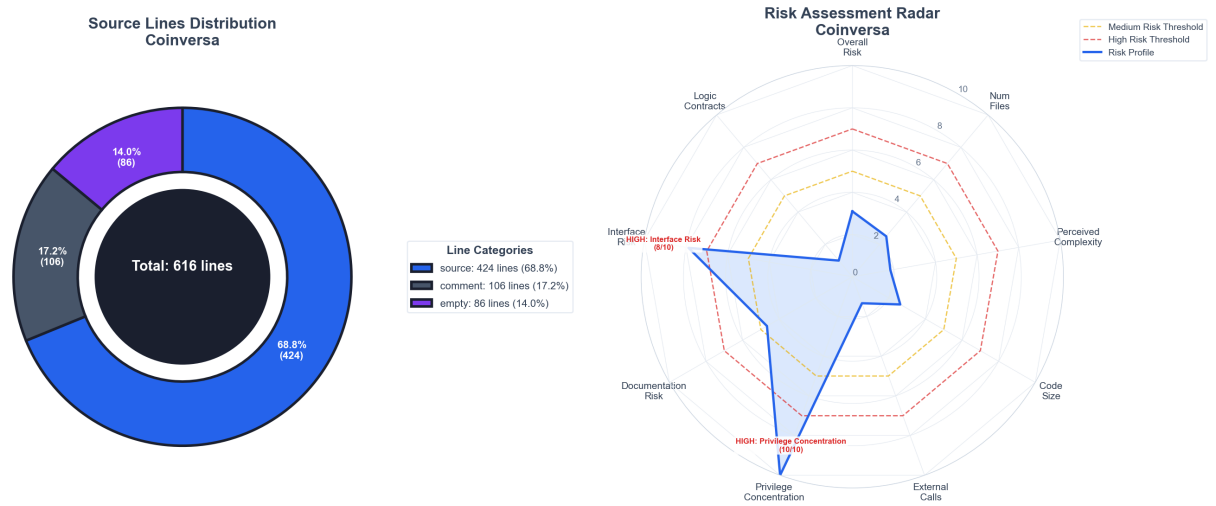
5.1 Tested Contract Files

Source repositories and commits for each contract in scope:

Contract	Source	Commit
DAML Contracts	https://github.com/Coinversaa/daml-contracts	82450d72ed81f2836a7c259234c413d6058e4fc5

File	Fingerprint (MD5)
./daml-contracts-main/daml/Coinversaa/TermsAcceptance.daml	66c2971afcea94f6c43dcf85213773ab
./daml-contracts-main/daml/Coinversaa/Transaction.daml	fa40ce090622b6d4152074738d610423
./daml-contracts-main/daml/Coinversaa/Types.daml	374d6d06877c4cac3a6051906c97f4bf
./daml-contracts-main/daml/Coinversaa/UserIdentity.daml	22c1e39177c2dd5369753119e51d6439

5.2 Source Lines & Risk



5.3 Capabilities

Module	Functions	Public	Restricted	Key Roles
TermsAcceptance.TermsAcceptance	1	0	1	operator
Transaction.TransactionCommitment	3	0	3	operator
Transaction.UserStatistics	2	0	2	operator
UserIdentity.UserIdentity	5	0	5	operator, userParty

5.4 Dependencies / External Imports

Import Path	Version
Coinversaa.UserIdentity	-
Coinversaa.Types	-

5.5 Source Units in Scope

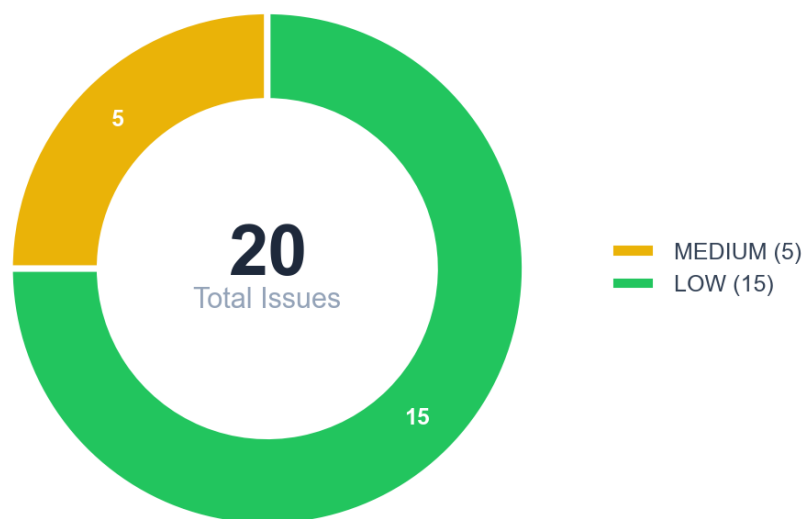
File	Instructions	Functions	Lines	nSLOC	Comments
Transaction.daml	5	5	254	178	42
Types.daml	0	4	136	105	17
TermsAcceptance.daml	1	1	78	51	14
UserIdentity.daml	5	6	148	90	33
Total	11	16	616	424	106

6. Scope of Work

This audit covers the Coinversa Canton DAML smart contracts that implement the platform's on-ledger identity and reconciliation layer. The contracts manage user identity (wallet linking, authentication keys), transaction commitment tracking, terms acceptance with cryptographic signatures, and user statistics aggregation. All contracts are deployed on the Canton Network distributed ledger.

6.1 Findings Overview

Findings are classified by severity: Critical, High, Medium, Low, and Informational.



No	Title	Severity	Status
6.2.1	Operator Controls UserIdentity Mutations That Should Belong to the User	MEDIUM	FIXED
6.2.2	Core Records Lack Enforced Uniqueness in the Current Canton 3 Design	MEDIUM	FIXED
6.2.3	User Cannot Revoke Their Own Terms Acceptance	MEDIUM	FIXED
6.2.4	Terms Acceptance Is Not Enforced by the Transaction Workflow	MEDIUM	FIXED

No	Title	Severity	Status
6.2.5	Operator-Supplied acceptedAt Timestamp Is Unconstrained	MEDIUM	FIXED
6.2.6	Wallet Address Validation Is Mostly Length-Based	LOW	FIXED
6.2.7	Transaction Hash Validation Accepts Invalid Pending and EVM Forms	LOW	FIXED
6.2.8	LinkWallet Does Not Verify Wallet Ownership	LOW	ACKNOWLEDGED
6.2.9	sourceWallet Is Not Validated Against walletType in TransactionCommitment	LOW	FIXED
6.2.10	termsHash Checks Length but Not Hex Content	LOW	FIXED
6.2.11	LinkAuthKey Allows Duplicate Authentication Keys	LOW	FIXED
6.2.12	Redeem Does Not Enforce a Transaction-State Transition	LOW	FIXED
6.2.13	ArchiveTransaction Allows Unilateral Record Deletion	LOW	ACKNOWLEDGED
6.2.14	Same-State UpdateStatus Calls Can Still Change blockNumber	LOW	ACKNOWLEDGED
6.2.15	Statistics Counters Are Not Consistently Monotonic	LOW	FIXED
6.2.16	Referrer Can Be Rewritten or Removed Without Constraint	LOW	ACKNOWLEDGED
6.2.17	UserStatistics Is Not Derived From Transaction Records	LOW	FIXED
6.2.18	signedMessage Is Not Bound to Wallet, Terms Version, or Terms Hash	LOW	FIXED
6.2.19	ipAddress Is Stored Directly On-Ledger	LOW	FIXED
6.2.20	Terms Versions Have No On-Ledger Lifecycle Management	LOW	FIXED

6.2 Manual and Automated Vulnerability Test

6.2.1 Operator Controls UserIdentity Mutations That Should Belong to the User

MEDIUM

FIXED

DESCRIPTION

UserIdentity is owned by `userParty`, but `LinkWallet`, `UnlinkWallet`, and `ArchiveIdentity` are all controlled by `operator`. This lets the operator mutate or destroy a user-owned identity record even though other identity-management choices in the same template (`LinkAuthKey`, `SetReferrer`) are correctly user-controlled.

CODE LOCATION

`daml/Coinversaa/UserIdentity.daml:8-36`, `71-89`, `107-143`

```
-- UserIdentity.daml:71-89
choice LinkWallet : ContractId UserIdentity
  with
    newWallet : WalletInfo
  controller operator    -- should be userParty
  do
    assertMsg "Invalid wallet address format" $
      validateWalletAddress newWallet.walletType newWallet.address
    assertMsg ("Maximum wallets reached (limit: " <> show maxLinkedWallets <> ")")
    $
      length linkedWallets < maxLinkedWallets
    now <- getTime
    create this with
      linkedWallets = newWallet :: linkedWallets
      updatedAt = now
```

RECOMMENDATION

Move those choices to `controller userParty`, or replace them with an operator-proposal and user-acceptance flow when operator involvement is still required.

6.2.2 Core Records Lack Enforced Uniqueness in the Current Canton 3 Design

MEDIUM

FIXED

DESCRIPTION

`TransactionCommitment`, `UserStatistics`, and `TermsAcceptance` contain commented-out key declarations for the uniqueness properties the system appears to expect. `UserIdentity` does not contain a matching active uniqueness mechanism either. Under Canton 3.x, contract keys are not supported with on-ledger uniqueness guarantees, so the code currently has no enforced deduplication path for these records.

CODE LOCATION

`daml/Coinversaa/Transaction.daml:47-50`, `186-188` | `daml/Coinversaa/TermsAcceptance.daml:31-34` | `daml/Coinversaa/UserIdentity.daml:19-25`

```
-- Transaction.daml:47-50
-- key (operator, txHash, chainId) : (Party, Text, Int)
-- maintainer key._1

-- Transaction.daml:186-188
-- key (operator, userParty) : (Party, Party)
-- maintainer key._1

-- TermsAcceptance.daml:31-34
-- key (operator, walletAddress, termsVersion) : (Party, Text, Text)
-- maintainer key._1
```

RECOMMENDATION

Treat uniqueness as an application-level responsibility. Add an off-ledger gateway that queries PQS or another authoritative index before any create operation, and only allows a dedicated creation actor to submit the create command when no logical duplicate already exists.

6.2.3 User Cannot Revoke Their Own Terms Acceptance

MEDIUM

FIXED

DESCRIPTION

The only mutating choice on `TermsAcceptance` is `RevokeAcceptance`, and it is controlled by `operator`. The user can query the record, but cannot change or revoke it.

CODE LOCATION

`daml/Coinversaa/TermsAcceptance.daml:7-29`, `64-68`

```
-- TermsAcceptance.daml:64-68
choice RevokeAcceptance : ()
  controller operator
  do
    return ()
```

RECOMMENDATION

Add a user-controlled revocation path. If the operator also needs an administrative revoke ability, keep both choices and document their separate purposes.

6.2.4 Terms Acceptance Is Not Enforced by the Transaction Workflow

MEDIUM

FIXED

DESCRIPTION

The reviewed DAML code does not link `TermsAcceptance` into `TransactionCommitment` creation or redemption. `Redeem` checks wallet membership through `UserIdentity`, but does not require a `TermsAcceptance` contract or any proof that the relevant terms version was accepted.

CODE LOCATION

`daml/Coinversaa/Transaction.daml:79-98` | `daml/Coinversaa/TermsAcceptance.daml:7-68`

```
-- Transaction.daml:79-98
choice Redeem : ContractId TransactionCommitment
  with
    redeemingUserIdentity : ContractId UserIdentity
  controller operator
  do
    user <- fetch redeemingUserIdentity
    assertMsg "Linked wallets in contract must match address"
      (any (\w -> w.address == sourceWallet) user.linkedWallets)
    assertMsg "Transactions can only be redeemed once"
      (isNone redeemer)
    now <- getTime
    let nowOpt = Some now
    let redeemingParty = Some user.userParty
    create this with
      redeemer = redeemingParty
      redemptionTime = nowOpt
```

RECOMMENDATION

Require a valid terms-acceptance reference when a transaction is created or redeemed, and validate that the acceptance belongs to the same wallet and to an active terms version.

6.2.5 Operator-Supplied `acceptedAt` Timestamp Is Unconstrained

MEDIUM

FIXED

DESCRIPTION

`acceptedAt` is a template field provided at creation time, but the `ensure` block does not validate it and no creation helper sets it from ledger time. Because `TermsAcceptance` is operator-signed, the operator can supply arbitrary timestamps.

CODE LOCATION

`daml/Coinversaa/TermsAcceptance.daml:24-29`, `37-42`

```
-- TermsAcceptance.daml:37-42
ensure
  validateWalletAddress walletType walletAddress &&
  termsVersion /= "" &&
  Text.length termsHash == 64 &&
  signature /= "" &&
  signedMessage /= ""
-- acceptedAt is NOT validated
```

RECOMMENDATION

Set `acceptedAt` from `getTime` inside a controlled creation choice, or explicitly validate user-supplied times against ledger time with an allowed skew window.

6.2.6 Wallet Address Validation Is Mostly Length-Based

LOW

FIXED

DESCRIPTION

`validateWalletAddress` checks only minimal shape constraints. EVM requires `0x` plus total length 42, but does not verify hexadecimal characters. Solana, Bitcoin, and Cosmos branches check only length ranges, with no Base58, Bech32, or network-prefix validation.

CODE LOCATION

`daml/Coinversaa/Types.daml:68-88`

```
validateWalletAddress walletType addr = case walletType of
  EVM ->
    Text.length addr == 42 &&
    Text.isPrefixOf "0x" addr
  Solana ->
    let len = Text.length addr
    in len >= 32 && len <= 44
  Bitcoin ->
    let len = Text.length addr
    in len >= 26 && len <= 62
  Cosmos ->
    let len = Text.length addr
    in len >= 39 && len <= 45
```

RECOMMENDATION

Add proper per-chain format checks, including character-set validation and required prefixes/HRPs.

6.2.7 Transaction Hash Validation Accepts Invalid Pending and EVM Forms

LOW

FIXED

DESCRIPTION

`validateTxHash` accepts any `pending\_`, `solana\_pending\_`, or `hl\_` prefix with no remainder bounds, and accepts 66-character `0x` hashes without verifying that the payload is hexadecimal.

CODE LOCATION

`daml/Coinversaa/Types.daml:91-100`

```
validateTxHash hash =
  Text.isPrefixOf "pending_" hash ||
  Text.isPrefixOf "solana_pending_" hash ||
  Text.isPrefixOf "hl_" hash ||
  (Text.length hash == 66 && Text.isPrefixOf "0x" hash) ||
  (Text.length hash >= 64 && Text.length hash <= 88)
```

RECOMMENDATION

Enforce length bounds for pending placeholders and hexadecimal validation for EVM-form hashes.

6.2.8 LinkWallet Does Not Verify Wallet Ownership

LOW

ACKNOWLEDGED

DESCRIPTION

`LinkWallet` checks formatting and wallet-count limits, but never proves that the user controls the wallet being attached.

CODE LOCATION

`daml/Coinversaa/UserIdentity.daml:71-89`

```
choice LinkWallet : ContractId UserIdentity
  with
    newWallet : WalletInfo
    controller operator
  do
    assertMsg "Invalid wallet address format" $
```



```

        validateWalletAddress newWallet.walletType newWallet.address
        assertMsg ("Maximum wallets reached (limit: " <> show maxLinkedWallets <> ")")
    $
        length linkedWallets < maxLinkedWallets
        now <- getTime
        create this with
            linkedWallets = newWallet :: linkedWallets
            updatedAt = now

```

RECOMMENDATION

Require a signed challenge, oracle attestation, or equivalent ownership proof before linking a wallet.

6.2.9 sourceWallet Is Not Validated Against walletType in TransactionCommitment

LOW

FIXED

DESCRIPTION

The `TransactionCommitment` ensure clause validates `txHash`, numeric fields, token symbols, and `dataHash`, but never validates `sourceWallet`.

CODE LOCATION

`daml/Coinversaa/Transaction.daml:18-20`, `53-65`

```

-- Transaction.daml:53-65
ensure
    validateTxHash txHash &&
    chainId >= 0 &&
    blockNumber >= 0 &&
    tokenInSymbol /= "" && tokenOutSymbol /= "" &&
    encryptedData /= "" &&
    Text.length dataHash == 64
-- sourceWallet is NOT validated

```

RECOMMENDATION

Add `validateWalletAddress walletType sourceWallet` to the ensure block.

6.2.10 termsHash Checks Length but Not Hex Content

LOW

FIXED

DESCRIPTION

`TermsAcceptance` only checks `Text.length termsHash == 64`. That enforces size, not SHA-256 hex encoding.

CODE LOCATION

`daml/Coinversaa/TermsAcceptance.daml:37-42`

```
ensure
  validateWalletAddress walletType walletAddress &&
  termsVersion /= "" &&
  Text.length termsHash == 64 && -- length only, no hex check
  signature /= "" &&
  signedMessage /= ""
```

RECOMMENDATION

Replace the length-only check with a reusable SHA-256 hex validator.

6.2.11 LinkAuthKey Allows Duplicate Authentication Keys

LOW

FIXED

DESCRIPTION

`LinkAuthKey` prepends a new `(authKey, encryptedKey)` pair but does not check whether the same key is already present.

CODE LOCATION

`daml/Coinversaa/UserIdentity.daml:24-25`, `51-68`

```
choice LinkAuthKey : ContractId UserIdentity
  with
    authKey: Text
    encryptedKey: Text
  controller userParty
  do
    now <- getTime
    create this with
      authKeys = ( authKey, encryptedKey ) :: authKeys
      updatedAt = now
```

RECOMMENDATION

Reject duplicates in the choice body and add an auth-key deduplication invariant.

6.2.12 Redeem Does Not Enforce a Transaction-State Transition

LOW

FIXED

DESCRIPTION

`Redeem` checks wallet linkage and one-time redemption, but it does not validate the current `status` and does not change `status` after redemption.

CODE LOCATION

`daml/Coinversaa/Transaction.daml:79-98` | `daml/Coinversaa/Types.daml:18-23`

```
-- Types.daml:18-23
data TxStatus =
  Pending
  | Confirmed
  | Failed
  deriving (Eq, Show)

-- Redeem does NOT check status == Confirmed
-- Redeem does NOT set a post-redemption status
```

RECOMMENDATION

Require redemption from an allowed status and persist an explicit redeemed state.

6.2.13 ArchiveTransaction Allows Unilateral Record Deletion

LOW

ACKNOWLEDGED

DESCRIPTION

`ArchiveTransaction` is operator-controlled and unconditional. Because the user is not a stakeholder on `TransactionCommitment`, this lets the operator remove active transaction records without a user-side check or notice in the template design.

CODE LOCATION

`daml/Coinversaa/Transaction.daml:44-45`, `151-154`

```
choice ArchiveTransaction : ()
  controller operator
do
  return ()
```

RECOMMENDATION

Restrict archival with business preconditions, add user visibility, and preserve an auditable tombstone or archive log if deletion remains necessary.

6.2.14 Same-State UpdateStatus Calls Can Still Change blockNumber

LOW

ACKNOWLEDGED

DESCRIPTION

`UpdateStatus` permits `Pending -> Pending`, `Confirmed -> Confirmed`, and `Failed -> Failed`, while also allowing `updatedBlockNumber` to overwrite `blockNumber`.

CODE LOCATION

`daml/Coinversaa/Transaction.daml:101-123`

```
case (status, newStatus) of
  (Pending, Confirmed) -> True
  (Pending, Failed) -> True
  (Confirmed, Failed) -> False
  (Failed, Confirmed) -> False
  (Confirmed, Pending) -> False
  (Failed, Pending) -> False
  (Failed, Failed) -> True      -- same-state allowed
  (Confirmed, Confirmed) -> True -- same-state allowed
  (Pending, Pending) -> True   -- same-state allowed

create this with
  status = newStatus
  blockNumber = fromOptional blockNumber updatedBlockNumber
  -- blockNumber silently overwritten on same-state transition
```

RECOMMENDATION

Require a genuine status transition before changing `blockNumber`, or split status changes and metadata updates into separate workflows.

6.2.15 Statistics Counters Are Not Consistently Monotonic

LOW

FIXED

DESCRIPTION

`newTotalTransactions` must be greater than or equal to the current value, but `newTotalVolumeUSD` is only checked for non-negativity and `newUniqueTokens` is not checked against the previous value.

CODE LOCATION

`daml/Coinversaa/Transaction.daml:190-195`, `198-226`

```
-- ensure block
totalTransactions >= 0 &&
totalVolumeUSD >= 0.0 &&
uniqueTokensTraded >= 0 &&
favoriteChain >= 0

-- UpdateStatistics
assertMsg "Transaction count cannot decrease" $
  newTotalTransactions >= totalTransactions

assertMsg "Volume cannot be negative" $
  newTotalVolumeUSD >= 0.0    -- not monotonic, just non-negative

-- newUniqueTokens has no check against previous value
```

RECOMMENDATION

Apply monotonic checks to all cumulative counters.

6.2.16 Referrer Can Be Rewritten or Removed Without Constraint

LOW

ACKNOWLEDGED

DESCRIPTION

`SetReferrer` accepts `Optional Party` and only checks that the referrer is not the same as `userParty`. The current code permits unlimited changes and explicit removal via `None`.

CODE LOCATION

`daml/Coinversaa/UserIdentity.daml:91-104`

```
choice SetReferrer : ContractId UserIdentity
  with
    newReferrer : Optional Party
  controller userParty
  do
    assertMsg "Referrer must be different from user" $
      if (isSome newReferrer) then (fromSome newReferrer) /= userParty else True
```

```
now <- getTime
create this with
  referrer = newReferrer
  updatedAt = now
```

RECOMMENDATION

Make the referrer immutable after first set, or add explicit cooldown and history semantics if changes are intended.

6.2.17 UserStatistics Is Not Derived From Transaction Records

LOW

FIXED

DESCRIPTION

`UpdateStatistics` accepts aggregate values and user-preference fields directly. It does not read or reference `TransactionCommitment`, so the statistics record is not mechanically tied to source transactions.

CODE LOCATION

`daml/Coinversaa/Transaction.daml:170-244`

RECOMMENDATION

Derive statistics from explicit transaction references or from an auditable off-ledger aggregation service with provenance.

6.2.18 signedMessage Is Not Bound to Wallet, Terms Version, or Terms Hash

LOW

FIXED

DESCRIPTION

`signedMessage` is required to be non-empty, but the code never checks whether it corresponds to `walletAddress`, `termsVersion`, or `termsHash`.

CODE LOCATION

`daml/Coinversaa/TermsAcceptance.daml:20-22`, `37-51`

```
ensure
  validateWalletAddress walletType walletAddress &&
  termsVersion /= "" &&
  Text.length termsHash == 64 &&
  signature /= "" &&
```

```
signedMessage /= "" -- non-empty only, no content binding

nonconsuming choice VerifyAcceptance : Bool
  with
    checkWallet : Text
    checkVersion : Text
    controller userParty
  do
    return $ walletAddress == checkWallet && termsVersion == checkVersion
    -- signedMessage is never verified
```

RECOMMENDATION

Enforce a canonical message format that includes the wallet, terms version, terms hash, and a replay-resistant nonce or domain separator.

6.2.19 ipAddress Is Stored Directly On-Ledger

LOW

FIXED

DESCRIPTION

`TermsAcceptance` stores `ipAddress : Optional Text` directly in the contract payload, which means the value becomes part of immutable ledger history once created.

CODE LOCATION

`daml/Coinversaa/TermsAcceptance.daml:24-26`

RECOMMENDATION

Keep raw IP data off-ledger. If a persistent reference is needed, store only a salted hash or an opaque external record identifier.

6.2.20 Terms Versions Have No On-Ledger Lifecycle Management

LOW

FIXED

DESCRIPTION

`termsVersion` is any non-empty string, and there is no registry, deprecation flow, or active-version check. The transaction workflow also does not verify that a user accepted the currently active version.

CODE LOCATION

`daml/Coinversaa/TermsAcceptance.daml:16-18`, `37-42` | `daml/Coinversaa/Transaction.daml:79-98`

RECOMMENDATION

Add a terms-version registry or equivalent active-version workflow, and gate transactions on acceptance of an active version.

7. Executive Summary

Two independent softstack experts performed an unbiased and isolated audit of the smart contract provided by the Coinversa team. The main objective of the audit was to verify the security and functionality claims of the smart contract. The audit process involved a thorough manual code review and automated security testing.

Overall, the audit identified a total of 20 issues, classified as follows:

- No critical issues were found
- No high issues were found
- 5 medium severity issues were found
- 15 low severity issues were found
- No informational issues were found

The Coinversa team has successfully addressed all identified issues from the audit. All vulnerabilities have been mitigated based on the recommendations provided in the report. A follow-up review confirms that the fixes have been implemented effectively, ensuring the security and functionality of the smart contract.

8. About the Auditor

Established in 2017 under the name Chainsulting, and rebranded as softstack GmbH in 2023, softstack has been a trusted name in Web3 Security space. Within the rapidly growing Web3 industry, softstack provides a comprehensive range of offerings that include software development, cybersecurity, and consulting services. Softstack's competency extends across the security landscape of prominent blockchains like Solana, Tezos, TON, Ethereum and Polygon. The company is widely recognized for conducting thorough code audits aimed at mitigating risk and promoting transparency.

The firm's proficiency lies particularly in assessing and fortifying smart contracts of leading DeFi projects, a testament to their commitment to maintaining the integrity of these innovative financial platforms. To date, softstack plays a crucial role in safeguarding over \$100 billion worth of user funds in various DeFi protocols.

Underpinned by a team of industry veterans possessing robust technical knowledge in the Web3 domain, softstack offers industry-leading smart contract audit services. Committed to evolving with their clients' ever-changing business needs, softstack's approach is as dynamic and innovative as the industry it serves.

Check our website for further information: <https://softstack.io>

9. Glossary

Term	Definition
CVSS	Common Vulnerability Scoring System
Canton	A privacy-enabled distributed ledger protocol by Digital Asset
DAML	Digital Asset Modeling Language — a smart contract language for distributed ledgers
EVM	Ethereum Virtual Machine
HRP	Human-Readable Part (Bech32 address prefix)
IDOR	Insecure Direct Object Reference
PQS	Participant Query Store — Canton's off-ledger query interface